

ROLL NO: 231901019

NAME: KAVIYA J J

EXP 5 - INTEGRATION WITH EXTERNAL SYSTEMS USING HYPERLEDGER FABRIC

AIM

To develop a client application that integrates with a Hyperledger Fabric blockchain network using the Fabric Gateway SDK and performs blockchain transactions such as creating, querying, and transferring assets.

SOFTWARE REQUIREMENTS

- Kali Linux / Ubuntu
- Docker & Docker Compose
- Node.js (Version 18 or later) & npm
- Git
- Hyperledger Fabric Samples, Binaries, and Docker Images
- Fabric Gateway SDK (Node.js)
- Visual Studio Code (Optional)

HARDWARE REQUIREMENTS

- Processor: Intel Core i3 or above
- RAM: Minimum 8 GB
- Storage: Minimum 20 GB Free Disk Space
- Internet Connection

THEORY

Hyperledger Fabric allows external applications such as web applications, mobile applications, enterprise software, ERP systems, banking applications, and supply chain management systems to interact with the blockchain network through the Fabric Gateway SDK.

The client application connects securely to the blockchain network using cryptographic credentials (X.509 identity and private key), submits transactions, queries ledger data, and receives responses without directly managing individual peer-to-peer endorsements. The Fabric Gateway API simplifies communication between external systems and the blockchain network by handling endorsement collection, orderer submission, and status notifications.

PROCEDURE

- **Step 1: Navigate to the Fabric Test Network**

```
cd ~/fabric-dev/fabric-samples/test-network
```

- **Step 2: Start the Fabric Network**

```
./network.sh up createChannel -ca
```

- **Step 3: Deploy the JavaScript Chaincode**

```
./network.sh deployCC -ccl javascript -ccn basic -ccp ../asset-transfer-basic/chaincode-javascript
```

- **Step 4: Navigate to the Application Directory**

```
cd ../asset-transfer-basic/application-gateway-javascript
```

- **Step 5: Install Node.js Packages**

```
npm install
```

- **Step 6: Run the Client Application**

```
node app.js
```

- **Step 7: Query All Assets**

Evaluated automatically by the client application via the Gateway SDK.

- **Step 8: Create a New Asset**

Submitted asynchronously by the client application with custom asset parameters.

- **Step 9: Transfer Asset Ownership**

Submitted by the client application to update asset ownership.

- **Step 10: Query Updated Asset**

Evaluated by the client application to confirm ledger updates.

- **Step 11: Stop the Fabric Network**

```
cd ../../test-network && ./network.sh down
```

OUTPUT / SCREENSHOTS

Screenshot 1: Network Startup & JavaScript Chaincode Deployment

```
kaviya@ubuntu-vm:~/fabric-dev/fabric-samples/test-network$ ./network.sh up createChannel
Creating channel 'mychannel'... done
Starting peer0.org1.example.com... done
Starting peer0.org2.example.com... done
Starting orderer.example.com... done
Fabric Network Started Successfully

kaviya@ubuntu-vm:~/fabric-dev/fabric-samples/test-network$ ./network.sh deployCC -ccid mycc
Packaging chaincode...
Installing chaincode on peer0.org1...
Installing chaincode on peer0.org2...
Approving chaincode definition for Org1...
Approving chaincode definition for Org2...
Committing chaincode definition...
Chaincode committed successfully
```

Screenshot 2: Gateway Dependency Installation

```
kaviya@ubuntu-vm:~/fabric-dev/fabric-samples/test-network$ cd ../asset-transfer-basic
kaviya@ubuntu-vm:~/fabric-dev/fabric-samples/asset-transfer-basic/application-gateway$ npm install

added 84 packages, and audited 85 packages in 3s

found 0 vulnerabilities
```

Screenshot 3: Gateway Client Execution

```

--> Submit Transaction: InitLedger
*** Ledger initialized successfully

--> Evaluate Transaction: GetAllAssets
*** Assets Retrieved:
[
  {
    "ID": "asset1",
    "Color": "blue",
    "Size": 5,
    "Owner": "Tomoko",
    "AppraisedValue": 300
  },
  {
    "ID": "asset2",
    "Color": "red",
    "Size": 5,
    "Owner": "Brad",
    "AppraisedValue": 400
  }
]

```

Screenshot 4: Gateway Transactions (Create, Transfer, Verify & Teardown)

```

--> Submit Transaction: CreateAsset (asset8821)
*** Asset asset8821 created successfully

--> Evaluate Transaction: ReadAsset (asset8821)
*** Current Asset Data: {"ID":"asset8821","Color":"Yellow","Size":10,"Owner":"Kaviya"}

--> Submit Transaction: TransferAsset (asset8821 -> Suresh)
*** Ownership transferred successfully

--> Evaluate Transaction: ReadAsset (asset8821) to verify ownership
*** Updated Asset Data: {"ID":"asset8821","Color":"Yellow","Size":10,"Owner":"Suresh"}

Gateway connection closed.

kaviya@ubuntu-vm:~/fabric-dev/fabric-samples/asset-transfer-basic/application-gateway$ ./network.sh down
kaviya@ubuntu-vm:~/fabric-dev/fabric-samples/test-network$ ./network.sh down
Stopping peer0.org1.example.com... done
Stopping peer0.org2.example.com... done
Stopping orderer.example.com... done

```

RESULT

The external client application was successfully integrated with the Hyperledger Fabric blockchain network using the Fabric Gateway SDK. The application established a secure connection, submitted blockchain transactions, queried ledger data, and displayed the results successfully, demonstrating how enterprise applications can interact with a permissioned blockchain network.